



FOR JAVA DEVELOPERS MOVING TO NESTJS

Crashed. Nothing sent.

Spring Boot to NestJS, episode 26: background jobs and queues

Fast response is not the guarantee. Durable handoff is.

Five receipts handed to @Async or to a promise nobody awaited were gone after a crash: zero sent, on both stacks.

In a BullMQ queue, four waiting jobs survived at once and the one that was running came back after stall detection.

A job that recorded a charge and then died ran again: at least once. And a payment committed before a crash never reached the queue.

In scope

- @Async and unawaited promises
- BullMQ with @nestjs/bullmq
- Crashes, stalls, retries, concurrency
- The database-to-queue gap

Not in scope

- Spring Batch in depth
- Kafka (next episode)

Spring Batch with a job repository is a durable model; @Async is not.

01 The measurements

FIVE 2 s RECEIPTS, CRASH 1 s IN	@Async 0; bare promise 0; BullMQ 4, then the fifth after 62 s
CHARGE RECORDED, THEN THE PROCESS DIED	charged 2 times
A JOB THAT FAILS ONCE	default: 1 attempt, failed; attempts 3: sent
TEN 500 ms JOBS	concurrency 1: 5.1 s; 5: 1.1 s (Spring 5.3 s, 1.3 s)
COMMIT, THEN CRASH BEFORE queue.add	payment 1, receipt job 0

Real processes, SIGKILL.

02 Put the work in a queue

BullMQ stores jobs in Redis and a worker takes them. Set attempts and backoff; the default is one attempt, one job at a time.

```
receipts.controller.ts
```

```
await this.receipts.add('receipt', { id, ms }, { attempts: 3, backoff: { type: 'exponential',  
delay: 200 } });
```

```
receipts.processor.ts
```

```
@Processor('receipts', { concurrency: 5 })  
export class ReceiptsProcessor extends WorkerHost {  
  async process(job: Job<{ id: string }>) {  
    // send the receipt: safe to run twice  
  }  
}
```

crash 1 s in: four waiting jobs sent within 12 s; the running one after stall detection

Run on a real machine, 2026-10-01, by `scripts/verify-queues.sh` in the course repository.

03 The gap before the queue

A durable queue protects work after it enters the queue. Writing the payment and adding the job are two systems; write an outbox row in the same transaction and relay it.

```
the outbox, as a concept
```

```
BEGIN;  
INSERT INTO payments (id) VALUES ('pay_1');  
INSERT INTO outbox (topic, payload) VALUES ('receipt', '{"id":"pay_1"}');  
COMMIT;  
-- a relay reads the outbox and adds the job
```

commit then crash before `queue.add`: one payment, no receipt job

Run on a real machine, 2026-10-01, by `scripts/verify-queues.sh` in the course repository.

Before you trust background work

1. Does it survive a crash, or live only in memory?
2. Can the job run twice safely?
3. How many attempts, with what backoff?
4. How many jobs at once?
5. What happens to a job that was running when the process died?
6. How does the job get from your database into the queue?

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-queues.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS queues https://docs.nestjs.com/techniques/queues

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. Redis is a trademark of Redis Ltd. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.